

ATIVIDADE • MATERIAL DO ALUNO

Vitrine Alegre

Front-end de e-commerce em React, construído com apoio de IA, a partir de um layout pronto e de uma API pública de produtos

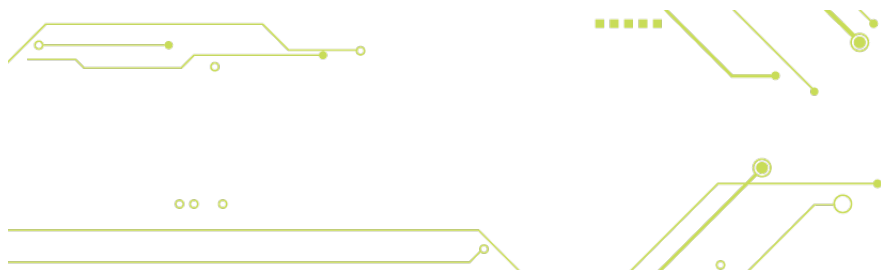
ATIVIDADE PRÁTICA

Desenvolvimento Front-End II

Prof. Cleziel Franzoni da Costa



Instituto Federal do Espírito Santo — Campus de Alegre
Tecnologia em Análise e Desenvolvimento de Sistemas • Presencial • 2026/2



Sumário

1 A proposta

- 1.1 O que precisa existir no final

2 As telas

- 2.1 Tela 1 — Vitrine (rota /)
- 2.2 Tela 2 — Detalhe do produto (rota /produtos/:id)
- 2.3 Tela 3 — Carrinho (rota /carrinho)
- 2.4 Os quatro estados
- 2.5 Comportamento responsivo
- 2.6 Mapa de componentes
- 2.7 Guia visual

3 A API de produtos

- 3.1 O que a API devolve
- 3.2 Decisões já tomadas — não invente outras

4 Requisitos

- 4.1 Rotas
- 4.2 Núcleo obrigatório
- 4.3 Extensões — escolha pelo menos duas

5 Trabalhando com IA sem virar refém dela

- 5.1 O ciclo
- 5.2 As cinco partes de um prompt que funciona
- 5.3 Mais três prompts que rendem nesta atividade
- 5.4 Prompts que dão errado
- 5.5 Os erros que a IA comete especificamente aqui
- 5.6 Oito perguntas antes de aceitar qualquer resposta

6 Roteiro sugerido

- 6.1 Etapa 0 — nascer o projeto
- 6.2 Etapa 1 — a camada de serviços, antes de qualquer tela
- 6.3 Etapa 2 — um cartão, estático
- 6.4 Etapa 3 — a lista de verdade, com os quatro estados
- 6.5 Etapa 4 — busca, categoria, ordenação e paginação
- 6.6 Etapa 5 — a página de detalhe
- 6.7 Etapa 6 — o carrinho
- 6.8 Etapa 7 — as três larguras
- 6.9 Etapa 8 — o que fica depois

7 Os dois arquivos de registro

- 7.1 PROMPTS.md
- 7.2 DIARIO-DA-IA.md

8 Checklist antes de considerar pronto

9 Erros comuns: diagnóstico rápido

■ DICA DO PROFESSOR — O que você recebe pronto e o que você constrói

- **Pronto:** seis imagens com o site inteiro desenhado — vitrine, detalhe, carrinho, estados, versão mobile e o mapa de componentes;
- **Pronto:** a API de produtos, pública, gratuita e sem cadastro;
- **Seu:** todo o código React — componentes, estado, rotas, serviços;
- **Seu:** todo o CSS. Nenhuma linha de estilo é entregue. As imagens são a especificação; traduzi-las em CSS é parte da atividade;
- **Seu:** o registro honesto de como você usou a IA para chegar lá.

■ 1. A proposta

A loja **Vitrine Alegre** já foi desenhada. O layout está fechado, a API está no ar, o produto está definido. Falta o front-end — e é aí que você entra, **conversando com uma IA** (Claude, ChatGPT, Gemini, Copilot: escolha a sua) para escrever o código React que transforma o desenho em site funcionando.

Esta atividade não é sobre digitar mais rápido. É sobre **dirigir** uma ferramenta que escreve código muito rápido e erra com a mesma velocidade. Quem sabe React usa a IA e ganha uma tarde; quem não sabe, usa a IA e ganha um projeto que roda até a primeira pergunta.

■ ATENÇÃO — A regra que muda tudo

A IA escreve, mas **você assina**. Qualquer linha do projeto pode virar pergunta: por que este `useEffect` tem esse array de dependências? Por que a `key` é o `id` e não o índice? O que acontece se a rede cair no meio deste `fetch`?

"Foi a IA que fez" não é resposta — é a confissão de que o código não é seu. Código que você não sabe explicar deve ser **jogado fora e refeito**, não entregue.

1.1 O que precisa existir no final

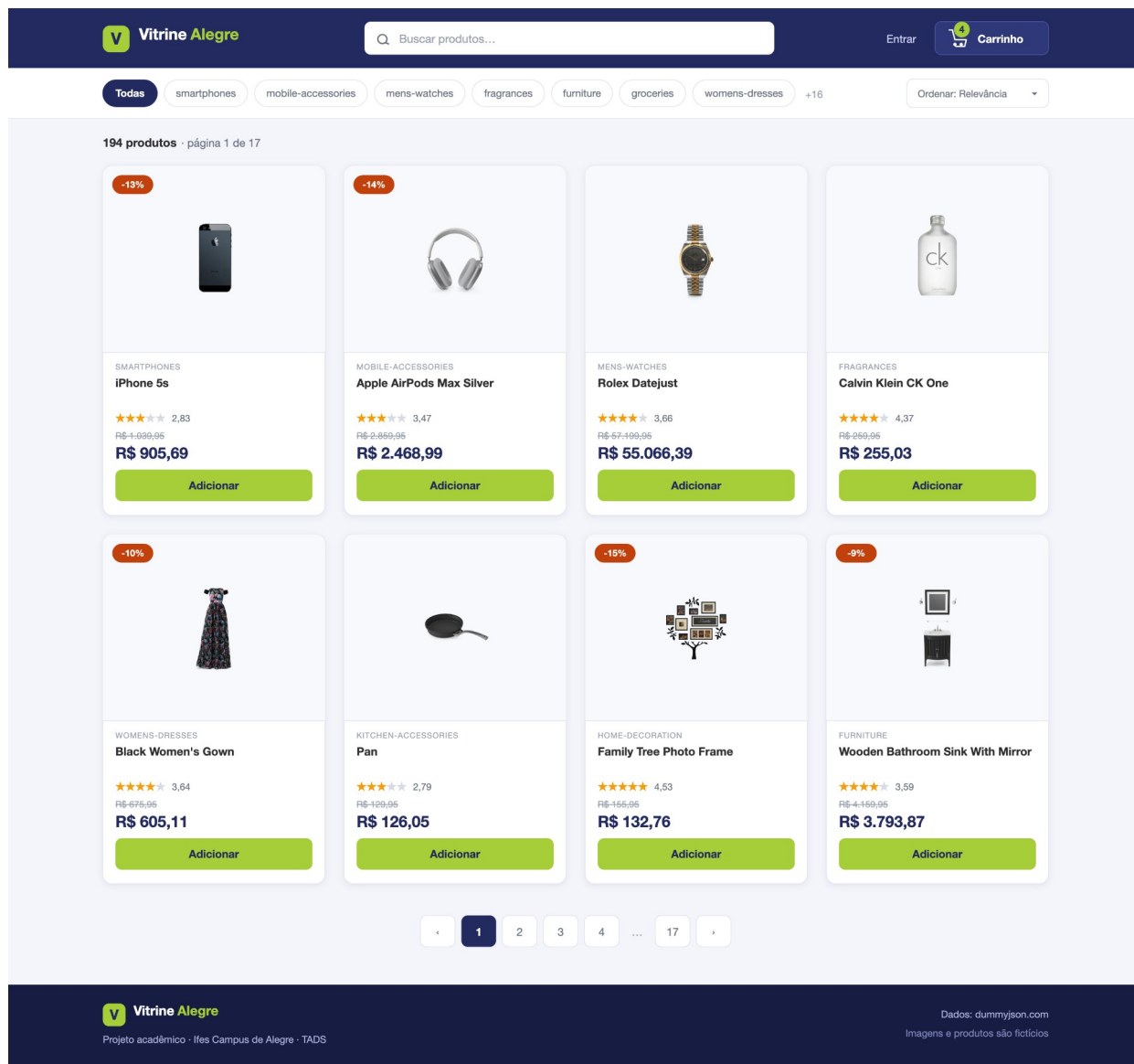
- Um projeto **React** (Vite) rodando com `npm run dev`, sem erros no console;
- As **três telas** do layout: vitrine, detalhe do produto e carrinho — navegáveis por rotas;
- Os **quatro estados** de interface em toda tela que fala com a rede;
- Um repositório Git com **commits ao longo do caminho** (não um commit único chamado "projeto");
- Um **README.md** que leva qualquer pessoa do zero ao app rodando;
- Um **PROMPTS.md** com as conversas que produziram o código;
- Um **DIARIO-DA-IA.md** com pelo menos **cinco erros** que a IA cometeu, como você percebeu e como corrigiu.

Prazo, formato de envio e composição em pontos ficam no **AVA** — este documento é a especificação técnica e vale enquanto o projeto existir.

2. As telas

As seis imagens abaixo estão em alta resolução na pasta **mockups/**, ao lado deste PDF. Abra-as no tamanho original: os detalhes (espaçamentos, pesos de fonte, hierarquia de preço) só aparecem lá. Elas são o contrato visual da atividade — e também são o melhor material que você pode dar à IA, porque a maioria das ferramentas aceita imagem no prompt.

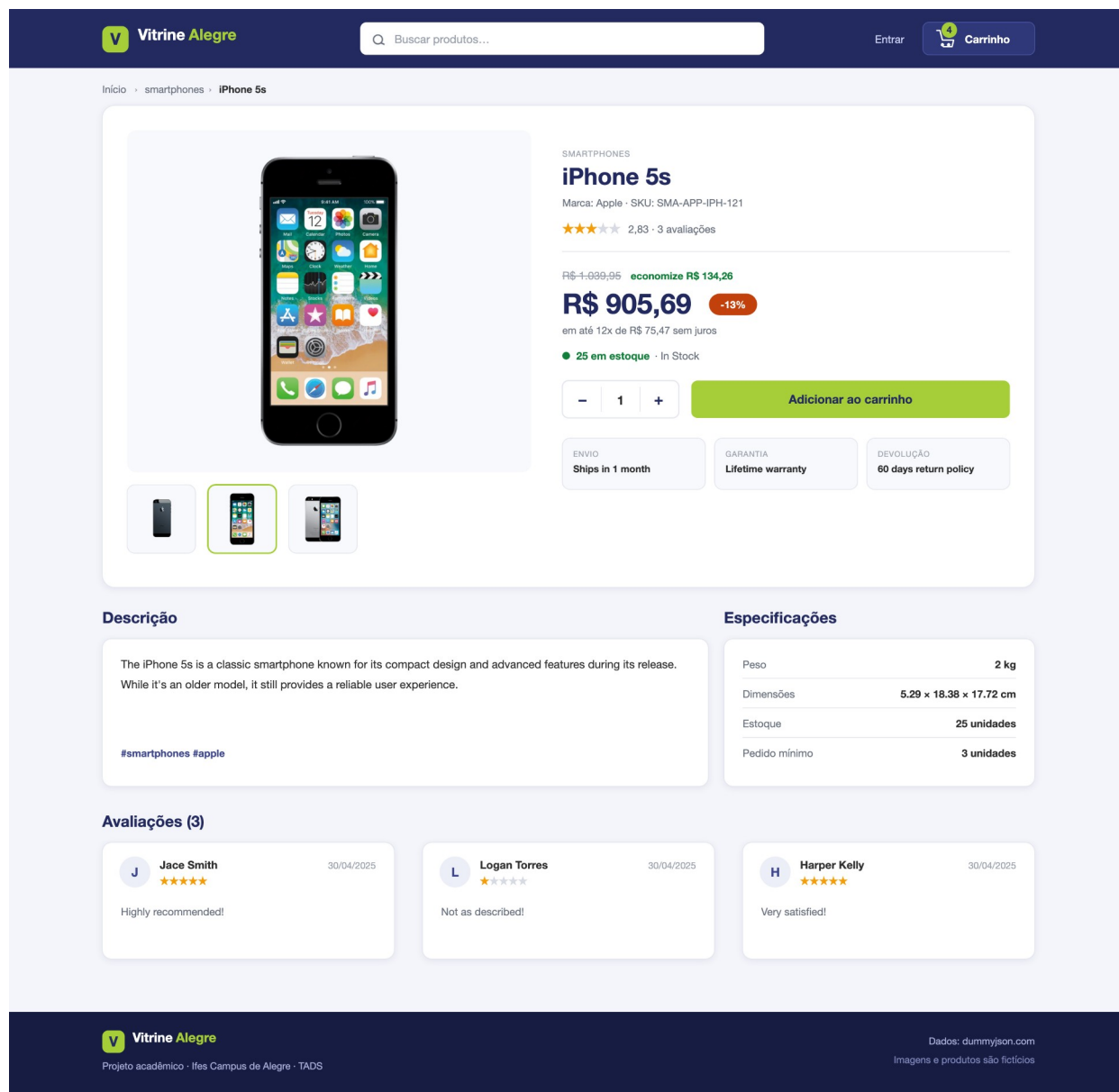
2.1 Tela 1 — Vitrine (rota /)



01-home-vitrine.png — grade de 4 colunas, filtro por categoria, busca, ordenação e paginação.

Cabeçalho fixo com busca e contador do carrinho; barra de categorias em pílulas, com a ativa preenchida; contagem de resultados; grade de cartões; paginação numerada. O cartão mostra selo de desconto quando o abatimento é relevante, categoria em caixa alta, título em no máximo duas linhas, estrelas com a nota, o preço antigo riscado, o preço final em destaque e o botão de adicionar.

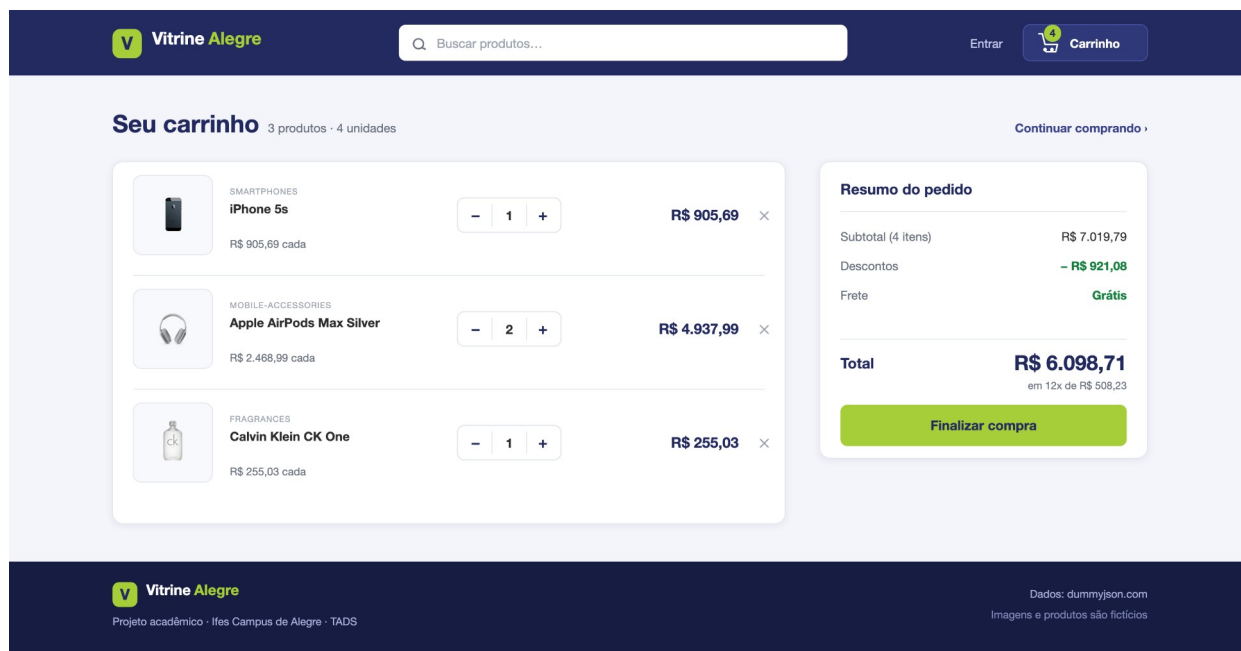
2.2 Tela 2 — Detalhe do produto (rota /produtos/:id)



02-detalle-produto.png — galeria, bloco de compra, descrição, especificações e avaliações.

Trilha de navegação no topo; galeria com imagem grande e miniaturas clicáveis (a ativa tem borda destacada); bloco de compra com preço, parcelamento, estoque, seletor de quantidade e o botão principal; três pílulas de informação vindas da API; descrição, tabela de especificações e as avaliações do produto.

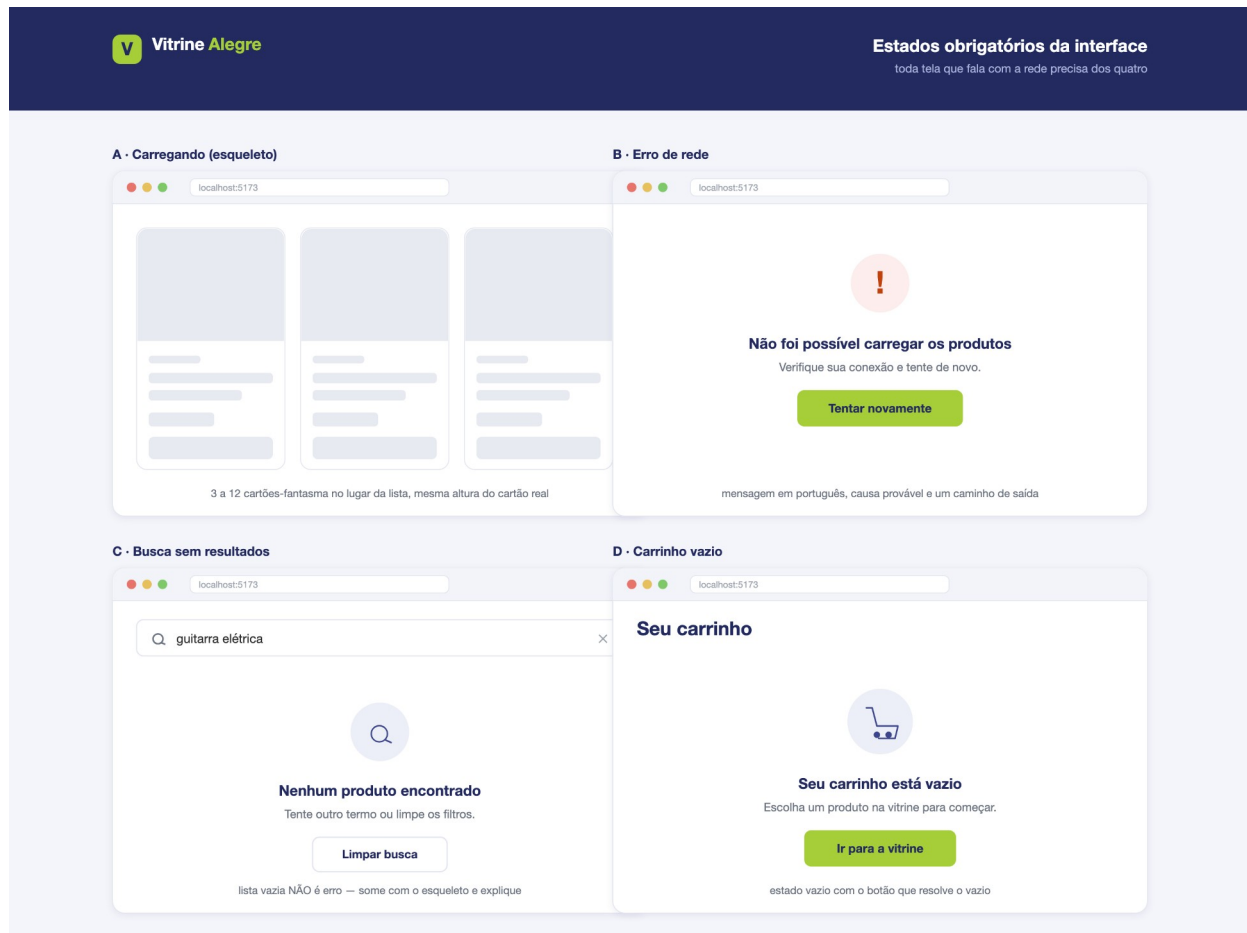
2.3 Tela 3 — Carrinho (rota /carrinho)



03-carrinho.png — lista de itens à esquerda, resumo do pedido à direita.

Cada linha tem miniatura, título, preço unitário, seletor de quantidade, subtotal da linha e o botão de remover. O resumo à direita soma tudo: subtotal pelos preços cheios, desconto acumulado, frete e total. **Todos esses números são derivados** — calcule-os a partir da lista de itens, nunca guarde cada um em seu próprio estado.

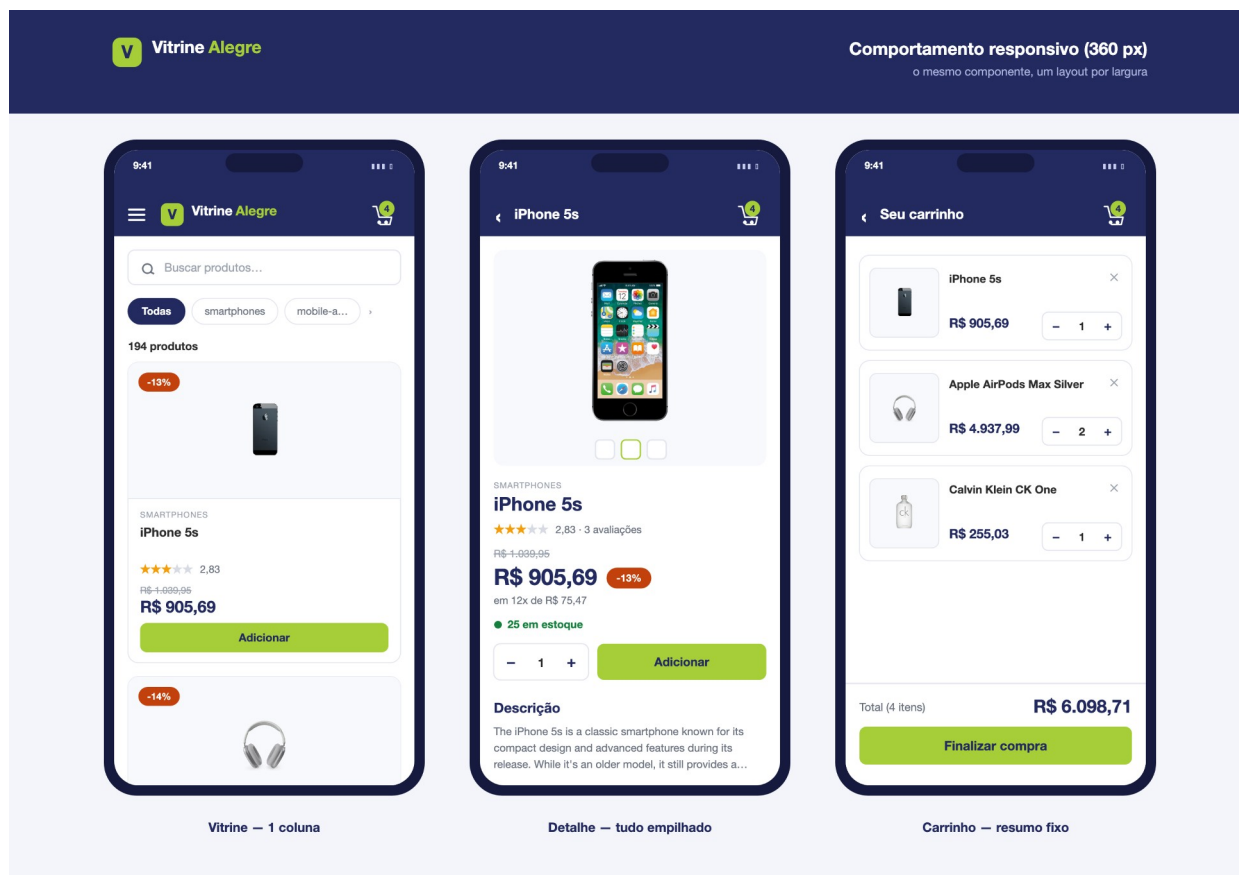
2.4 Os quatro estados



04-estados-da-interface.png — carregando, erro, lista vazia e carrinho vazio.

Esta é a imagem que separa o projeto de aula do projeto de portfólio. Toda tela que depende da rede tem **quatro caras**, e as quatro precisam existir: enquanto carrega, quando dá erro, quando o resultado é uma lista vazia e quando o usuário ainda não fez nada. **Lista vazia não é erro** — é resposta bem-sucedida com zero itens, e a tela precisa dizer isso.

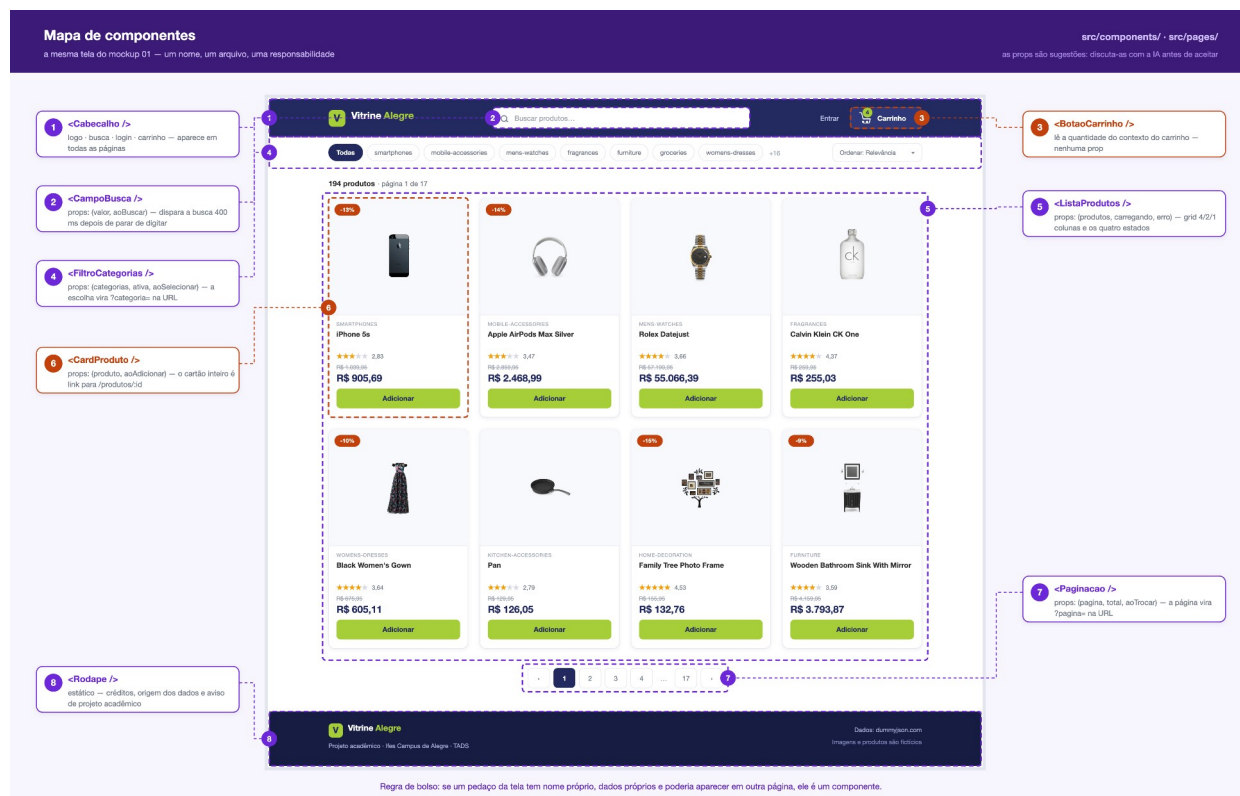
2.5 Comportamento responsivo



05-responsivo-mobile.png — as mesmas telas em 360 px de largura.

A grade vai de quatro colunas para duas e depois para uma; o cabeçalho encolhe e o menu vira ícone; no carrinho, o resumo desgruda da lateral e vira uma barra fixa no rodapé da tela. É o **mesmo componente** em todas as larguras — o que muda é o CSS, não a árvore de componentes.

2.6 Mapa de componentes



06-mapa-de-componentes.png — a vitrine recortada em componentes, com as props sugeridas.

Use este mapa como ponto de partida da conversa com a IA, não como ordem. As props são sugestões razoáveis; se você chegar a um recorte melhor, ótimo — **defenda-o**. A regra de bolso está no rodapé da imagem: se um pedaço da tela tem nome próprio, dados próprios e poderia aparecer em outra página, ele é um componente.

2.7 Guia visual

As cores e medidas abaixo são as usadas nas imagens. Elas existem para você **não perder tempo com o contagotas** — a tradução delas em CSS (variáveis, classes, módulos, o que você preferir) é sua.

Papel na interface	Cor	Onde aparece
Primária (marca)	#232A60	cabeçalho, títulos, preço final, pílula de categoria ativa
Primária escura	#171C42	rodapé
Ação / destaque	#A6CE39	botões principais, selo do carrinho, borda da miniatura ativa
Fundo da página	#F4F5FA	área atrás dos cartões
Superfície	#FFFFFF	cartões, campos, resumo do pedido
Borda	#E3E6EF	contorno de cartões, campos e divisórias
Texto	#2E2E2E	títulos de produto e corpo
Texto secundário	#6B7280 / #9CA3AF	categoria, preço riscado, legendas
Alerta / desconto	#C2410C	selo de desconto e ícone de erro
Sucesso	#15803D	estoque disponível, frete grátis, valor economizado
Avaliação	#F59E0B	estrelas preenchidas (as vazias em #D8DBE4)

Medida	Valor nas imagens
Largura do conteúdo	1200 px centralizados, com respiro de 120 px nas laterais
Grade de produtos	4 colunas · espaço de 24 px · 2 colunas em tablet · 1 em celular
Cartão de produto	282 × 444 px, canto de 12 px, borda de 1 px e sombra discreta
Botões	altura 40–48 px, canto de 8 px, rótulo em negrito
Campos e pílulas	altura 34–42 px; pílula de categoria com canto totalmente arredondado
Tipografia	família sem serifa do sistema; título de cartão 14,5 px · preço final 20 px · título da página 28–30 px
Cabeçalho / rodapé	76 px e 110 px de altura

■ DICA DO PROFESSOR — Que fidelidade é cobrada

Não é pixel a pixel. O que se olha é: hierarquia (o preço final salta mais que o riscado?), ritmo (os espaçamentos são consistentes ou cada cartão tem o seu?), alinhamento (as colunas fecham?) e **os quatro estados**.

Uma tela com fonte diferente e espaçamento coerente passa. Uma tela idêntica ao mockup que quebra ao carregar não passa.

3. A API de produtos

A loja consome a **DummyJSON**, uma API pública de produtos falsos: 194 produtos com imagem de verdade, 24 categorias, busca, ordenação e paginação. Sem cadastro, sem chave, sem limite prático — e com CORS liberado, então o navegador chama direto.

Para quê	Endpoint
Listar com paginação	https://dummyjson.com/products?limit=12&skip=0
Um produto	https://dummyjson.com/products/121
Buscar por texto	https://dummyjson.com/products/search?q=phone
Lista de categorias	https://dummyjson.com/products/category-list
Produtos de uma categoria	https://dummyjson.com/products/category/smartphones
Ordenar	.../products?sortBy=price&order=asc
Pedir só alguns campos	.../products?select=id,title,price,thumbnail

Antes de escrever qualquer código, **abra cada uma dessas URLs no navegador**. Você precisa ter visto a resposta com os próprios olhos para reconhecer quando a IA inventar um campo que não existe.

3.1 O que a API devolve

```
GET https://dummyjson.com/products?limit=2

{
  "products": [                                // ← a lista vem DENTRO de products
    {
      "id": 121,
      "title": "iPhone 5s",
      "description": "The iPhone 5s is a classic smartphone...",
      "category": "smartphones",
      "price": 199.99,                          // ← preço CHEIO, em dólar
      "discountPercentage": 12.91,             // ← desconto a aplicar
      "rating": 2.83,                          // ← número solto (na Fake Store é objeto)
      "stock": 25,
      "brand": "Apple",
      "tags": ["smartphones", "apple"],
      "thumbnail": "https://cdn.dummyjson.com/.../thumbnail.webp",
      "images": ["...1.webp", "...2.webp", "...3.webp"],
      "reviews": [ { "rating": 5, "comment": "...", "reviewerName": "..." } ],
      "shippingInformation": "Ships in 1 month",
      "warrantyInformation": "Lifetime warranty",
      "returnPolicy": "60 days return policy"
    }
  ],
  "total": 194,                                // ← total de produtos, não da página
  "skip": 0,
  "limit": 2
}
```

3.2 Decisões já tomadas — não invente outras

Questão	Decisão desta atividade
Preço final	$\text{price} \times (1 - \text{discountPercentage} \div 100)$. O price cru vira o valor riscado.

Questão	Decisão desta atividade
Moeda	a API cobra em dólar; a loja mostra em real com uma cotação fixa no código (<code>const COTACAO = 5.20</code>) e formatação por <code>Intl.NumberFormat('pt-BR', ...)</code> .
Selo de desconto	só aparece a partir de 5%.
Imagem	<code>thumbnail</code> na vitrine e no carrinho; o vetor <code>images</code> na galeria do detalhe.
Idioma dos dados	títulos, descrições e avaliações vêm em inglês e ficam em inglês. Traduz-se a interface , não o conteúdo da API.
Paginação	12 produtos por página; <code>skip = (página - 1) × 12</code> ; o número de páginas sai de <code>total</code> .
Carrinho	vive no navegador (contexto do React, opcionalmente espelhado no <code>localStorage</code>). Não existe endpoint de pedido.

■ ATENÇÃO — Quatro pegadinhas desta API (a IA cai em todas)

- **A lista não é o corpo da resposta.** `response.json()` devolve um objeto; os produtos estão em `.products`. Um `.map()` direto no objeto quebra na hora.
- **São dois endpoints de categoria.** `/products/category-list` devolve textos (`"smartphones"`); `/products/categories` devolve objetos (`{ slug, name, url }`). Modelos treinados na versão antiga da API confundem os dois e o filtro para de funcionar.
- **`total` é o total do catálogo**, não o tamanho da página — é dele que sai a quantidade de páginas.
- **A Fake Store API é outra coisa.** Se a IA sugerir `fakestoreapi.com`, o campo de imagem vira `image` (singular) e a nota vira `rating.rate`. Misturar as duas é o erro mais comum desta atividade.

Se a DummyJSON estiver fora do ar no dia, a alternativa aceita é a **Fake Store API** (<https://fakestoreapi.com/products>) — mas então o serviço precisa ser adaptado por inteiro, e isso vira registro no diário.¹

¹ Vale abrir o console de rede do navegador (aba Network) enquanto a página carrega: ver a requisição, o status e o corpo da resposta é o jeito mais rápido de descobrir se o problema é seu, da IA ou da API.

4. Requisitos

4.1 Rotas

Rota	Tela	O que precisa funcionar
/	Vitrine	lista paginada, busca, filtro por categoria, ordenação
/produtos/:id	Detalhe	carrega pelo id da URL; recarregar a página (F5) mantém o produto
/carrinho	Carrinho	itens, quantidades, remoção e resumo calculado
qualquer outra	404	mensagem e um caminho de volta para a vitrine

Busca, categoria e página vivem na **URL** (`/?busca=phone&categoria=smartphones&pagina=2`). É o que permite compartilhar um resultado e o que faz o botão "voltar" do navegador se comportar. Estado que só existe na memória some no F5.

4.2 Núcleo obrigatório

- Projeto **React + Vite**, com componentes em arquivos separados e pastas com nome (`components/`, `pages/`, `services/`);
- **Camada de serviços**: todo `fetch` mora em um arquivo só (`services/api.js`); nenhum componente conhece a URL da API;
- Vitrine com **busca** (que espera você parar de digitar antes de disparar), **filtro por categoria**, **ordenação** e **paginação**;
- Detalhe com galeria funcional (clique na miniatura troca a imagem grande);
- Carrinho com **contexto** compartilhado: adicionar da vitrine ou do detalhe, mudar quantidade, remover, e o contador do cabeçalho reagindo na hora;
- Totais **derivados** da lista de itens — subtotal, desconto e total nunca são estado próprio;
- **Carregando, erro, vazio e inicial** em toda tela que fala com a rede;
- Layout responsivo nas três larguras do mockup 05;
- Console do navegador **limpo**: sem avisos de `key`, sem erros de rede não tratados.

4.3 Extensões — escolha pelo menos duas

- Carrinho que **sobrevive ao F5** (`localStorage`), com o cuidado de ler uma vez e não a cada renderização;
- **Favoritos** com página própria;
- Um **hook próprio** (`useProdutos`, `useDebounce`, `useCarrinho`) documentado no README;
- **Cancelamento de requisição** com `AbortController` quando o usuário digita rápido ou troca de página;
- **Tema claro/escuro** com contexto e persistência;
- **Acessibilidade**: navegação por teclado no menu e na galeria, textos alternativos reais nas imagens, foco visível;
- **Deploy** na Vercel ou Netlify, com o link no README.

■ ATENÇÃO — Fora de escopo

Não há login, não há checkout de verdade, não há back-end seu e **não se instala biblioteca de componentes prontos** (Bootstrap, Material UI, Tailwind, shadcn). O CSS é escrito à mão — com ajuda da IA, mas escrito. Pedir para a IA "usar Tailwind" é justamente a saída que esta atividade fecha.

Bibliotecas permitidas: **react-router-dom** e, se quiser, **axios**. Mais nada sem combinar antes.

5. Trabalhando com IA sem virar refém dela

A diferença entre quem termina esta atividade em uma tarde e quem passa a semana caçando bug não está na ferramenta — as duas pessoas usam a mesma. Está no **tamanho do pedido** e na **leitura do que voltou**.

5.1 O ciclo

1. **Contextualize.** Diga a versão (React 18 ou 19, Vite, react-router-dom v6), mostre a árvore de arquivos, cole o JSON real que a API devolveu e anexe a imagem da tela. Sem isso a IA escreve para um projeto imaginário.
2. **Peça pequeno.** Um componente por vez. "Faça o e-commerce inteiro" devolve 400 linhas plausíveis que ninguém consegue depurar — inclusive você.
3. **Leia e rode.** Antes de colar: entendi cada linha? Depois de colar: abri no navegador, testei o caminho feliz, o erro e o vazio?
4. **Registre.** Prompt que funcionou vai para o **PROMPTS.md**; erro que apareceu vai para o **DIARIO-DA-IA.md**. Registrar depois nunca acontece.

5.2 As cinco partes de um prompt que funciona

Parte	Exemplo nesta atividade
Papel	"Você é uma pessoa desenvolvedora React sênior revisando o código de um aluno."
Contexto	versões, estrutura de pastas, o JSON real da API, a imagem da tela
Tarefa	"Escreva o componente CardProduto" — uma coisa só
Restrições	"componente de função, sem bibliotecas de UI, CSS em arquivo separado, nada de índice como key"
Formato	"devolva o arquivo completo e depois explique em 3 frases as decisões que tomou"

Você é uma pessoa desenvolvedora React sênior.

Contexto: projeto React 18 com Vite e react-router-dom v6. Estrutura:

src/components/ src/pages/ src/services/api.js

A API devolve exatamente isto (colei a resposta real de /products?limit=1):

```
{ "products": [ { "id": 121, "title": "iPhone 5s", "price": 199.99,
  "discountPercentage": 12.91, "rating": 2.83, "thumbnail": "..."} ],
  "total": 194, "skip": 0, "limit": 1 }
```

Anexei a imagem do cartão como ele deve ficar.

Tarefa: escreva SÓ o componente CardProduto.jsx.

Restrições:

- componente de função, sem classes;

- recebe o produto por props, não busca dados sozinho;
- preço final = price * (1 - discountPercentage/100), convertido a real pela constante COTACAO e formatado com Intl.NumberFormat('pt-BR');
- o selo de desconto só aparece a partir de 5%;
- o cartão inteiro é um <Link> para /produtos/:id;
- nenhuma biblioteca de UI; as classes CSS eu escrevo depois.

Formato: o arquivo completo e, depois, 3 frases explicando suas decisões.

5.3 Mais três prompts que rendem nesta atividade

// 1. Traduzir a imagem em CSS – descrevendo, não mandando adivinhar
Anexei o mockup da vitrine. As cores são #232A60 (marca), #A6CE39 (ação), #F4F5FA (fundo), #E3E6EF (borda). O cartão tem 282x444, canto de 12px. Escreva o CSS da grade e do cartão, com 4 colunas acima de 1024px, 2 entre 640 e 1024 e 1 abaixo. Use CSS puro com variáveis no :root. Explique por que escolheu grid ou flex.

// 2. Revisão adversarial – o prompt mais útil de todos
Aqui está o meu ListaProdutos.jsx. Aja como revisor cético: aponte vazamentos de memória, dependências erradas do useEffect, estado que poderia ser derivado e casos de erro não tratados. Para cada achado, diga como REPRODUZIR o problema no navegador. Não reescreva o arquivo.

// 3. Explicação sob medida – quando a resposta veio boa demais
Você me deu este useEffect. Explique linha a linha por que o array de dependências é esse, o que acontece se eu esvaziá-lo e o que a função de limpeza evita. Depois me dê UM exercício para eu testar se entendi.

5.4 Prompts que dão errado

O que se pede	O que volta	Por que dói
"Faça um e-commerce em React"	um arquivo gigante com dados inventados	não usa a API, não segue o layout e não dá para depurar
"Deixa igual à imagem" (sem cores nem medidas)	um layout genérico de tutorial	a IA não tem conta-gotas; dê os valores
"Está dando erro, conserta"	uma reescrita que troca o bug de lugar	sem a mensagem e o arquivo, ela adivinha
"Usa a melhor biblioteca pra isso"	Tailwind, MUI, Redux, react-query...	instala dependência que você não sabe manter — e está fora do escopo
Aceitar a primeira resposta longa	código plausível e não testado	o custo aparece três horas depois, no bug que ninguém entende

5.5 Os erros que a IA comete especificamente aqui

Cada linha cruza um modo de falha conhecido dessas ferramentas com o formato **real** desta API — as colunas do meio e da direita foram conferidas contra a resposta que a dummyjson.com devolve hoje. Não é lista genérica de "cuidados com IA": é o que quebra neste projeto.

O que a IA escreve	O que acontece	A correção
<code>data.map(...)</code> na resposta da lista	"data.map is not a function"	a lista está em <code>data.products</code>

O que a IA escreve	O que acontece	A correção
<code>produto.image</code>	imagem quebrada em todo cartão	aqui o campo é <code>thumbnail</code>
<code>produto.rating.rate</code>	erro de propriedade de undefined	na DummyJSON <code>rating</code> já é número
<code>useEffect</code> sem array de dependências	requisição em laço infinito (veja a aba Rede)	array vazio para "uma vez", ou as dependências reais
<code>key={index}</code> no map	campos e imagens trocando de lugar ao filtrar	<code>key={produto.id}</code>
<code>fetch</code> sem <code>response.ok</code>	erro 404 vira tela em branco	checar <code>response.ok</code> e lançar erro
<code>useEffect</code> sem função de limpeza	aviso de atualização em componente desmontado	<code>AbortController</code> ou um sinalizador <code>cancelado</code>
Total do carrinho em <code>useState</code>	total desatualizado depois de remover item	calcular na renderização a partir dos itens
<code><Switch></code> e <code>component={...}</code>	erro de import do react-router	v6 usa <code><Routes></code> e <code>element={<X />}</code>
<code>localStorage.getItem</code> no corpo do componente	leitura a cada renderização e piscar de tela	estado inicial preguiçoso ou <code>useEffect</code>
<code>class</code> no JSX e <code>!important</code> no CSS	estilo não aplica; CSS vira briga	<code>className</code> ; resolver especificidade, não empilhar <code>!important</code>
Componentes de classe e <code>componentDidMount</code>	código de 2017 no meio do projeto	peça explicitamente hooks e componentes de função

5.6 Oito perguntas antes de aceitar qualquer resposta

1. Este código usa a **API que eu escolhi**, com os campos que eu vi no navegador?
2. Consigo explicar **cada linha** — inclusive as que "só funcionam"?
3. Tem `useEffect`? Então: o array de dependências está certo e existe limpeza?
4. Tem `map`? Então a `key` é um **id estável**, não o índice?
5. Algum estado aqui poderia ser **calculado** a partir de outro?
6. O que essa tela mostra se a rede cair? E se a resposta vier vazia?
7. Instalou alguma biblioteca? Eu autorizei? Ela está no escopo?
8. Rodei no navegador **antes** de pedir a próxima coisa?

■ ATENÇÃO — O limite

Usar IA nesta atividade é **obrigatório** — o objetivo é justamente aprender a trabalhar com ela. O que não vale é entregar o que você não entende.

Um projeto menor, escrito devagar e explicável, vale mais do que um projeto completo que você viu nascer sem entender. Se faltar tempo, **corte extensão, não compreensão.**

6. Roteiro sugerido

Oito etapas, cada uma com um ponto de conferência. Faça um **commit** ao fim de cada uma: é assim que o histórico conta a história, e é assim que você tem para onde voltar quando a IA te levar para um beco.

6.1 Etapa 0 — nascer o projeto

```
npm create vite@latest vitrine-alegre -- --template react
# ele vai perguntar qual linter usar (Oxlint ou ESLint). Qualquer um serve;
# a flag --no-eslint responde "Oxlint" sozinha e pula a pergunta.

cd vitrine-alegre
npm install
npm install react-router-dom
npm run dev          # abre em http://localhost:5173

# limpe o que veio de exemplo: App.jsx, App.css e o conteúdo de index.css
# crie as pastas: src/components src/pages src/services src/context
```

O **create-vite** de hoje entrega **React 19**. Está tudo certo: nada nesta atividade muda entre 18 e 19. Se você quiser o mesmo ambiente dos projetos dos módulos, fixe **react** e **react-dom** em **18.3.1** no **package.json** antes do **npm install**.

✓ Ponto de conferência: a página padrão do Vite abre no navegador e o terminal não mostra erro.

6.2 Etapa 1 — a camada de serviços, antes de qualquer tela

Um arquivo **src/services/api.js** com funções que devolvem dados já prontos para a interface: **listarProdutos({ pagina, busca, categoria, ordenacao })**, **buscarProduto(id)**, **listarCategorias()**. Nenhum componente do projeto deve conhecer o endereço da API.

Peça à IA para escrever esse arquivo com verificação de **response.ok** e mensagens de erro em português. Depois **teste no navegador**, antes de existir qualquer componente: importe e chame a função no **App.jsx** com um **console.log**.

✓ Ponto de conferência: o console do navegador mostra os 12 primeiros produtos.

6.3 Etapa 2 — um cartão, estático

Copie do navegador um objeto de produto real e cole-o direto no componente. Sem rede, sem estado: só JSX e CSS até o cartão ficar igual ao do mockup — inclusive o selo de desconto, o preço riscado e as estrelas.

É aqui que nasce a maior parte do CSS do projeto. Vale gastar tempo: as medidas do guia visual estão na página anterior, e a IA transforma descrição em CSS muito melhor do que transforma "faz bonito".

✓ Ponto de conferência: o cartão isolado na tela é indistinguível do cartão do mockup 01.

6.4 Etapa 3 — a lista de verdade, com os quatro estados

Agora sim: **useEffect** chamando o serviço, **useState** para produtos, carregando e erro. Renderize os quatro estados do mockup 04 **antes** de mexer em filtro ou paginação — e teste os quatro de propósito: desligue o wi-fi, troque a URL da API por uma errada, force uma busca sem resultado.

✓ Ponto de conferência: os quatro estados aparecem sob demanda, e a aba Rede mostra **uma** requisição por carregamento — não um laço.

6.5 Etapa 4 — busca, categoria, ordenação e paginação

Tudo na URL. A busca só dispara depois que o usuário para de digitar (400 ms). Trocar de categoria ou de busca **volta para a página 1** — esse é o bug clássico desta etapa, e a IA raramente lembra dele sozinha.

✓ Ponto de conferência: copiar a URL, abrir em outra aba e cair exatamente no mesmo resultado.

6.6 Etapa 5 — a página de detalhe

Rota `/produtos/:id`, dados buscados pelo id da URL (nunca passados por navegação — F5 tem que funcionar), galeria com miniaturas, e os quatro estados de novo, agora para um produto só. Um id inexistente (`/produtos/99999`) precisa mostrar erro, não tela branca.

✓ Ponto de conferência: F5 na página de detalhe recarrega o mesmo produto; um id inválido mostra erro tratado.

6.7 Etapa 6 — o carrinho

Um contexto com a lista de itens e as ações de adicionar, mudar quantidade e remover. O contador do cabeçalho e a página do carrinho leem do mesmo lugar. Adicionar um produto que já está no carrinho **soma quantidade**, não duplica a linha.

Subtotal, desconto e total são calculados na renderização. Se você viu a IA criar um `useState` para o total, esse é um achado para o diário.

✓ Ponto de conferência: adicionar da vitrine e do detalhe, mudar quantidade, remover — e o contador do cabeçalho acompanhando.

6.8 Etapa 7 — as três larguras

Abra as ferramentas de desenvolvedor, ative o modo dispositivo e trabalhe em 360, 768 e 1440 px. A grade muda de colunas, o cabeçalho encolhe, o resumo do carrinho vira barra fixa. Nada de rolagem horizontal em nenhuma largura.

✓ Ponto de conferência: as três larguras do mockup 05, sem rolagem lateral e sem texto cortado.

6.9 Etapa 8 — o que fica depois

`README.md`, `PROMPTS.md` e `DIARIO-DA-IA.md` fechados, console limpo, `npm run build` sem erro. Se for fazer deploy, é aqui.

7. Os dois arquivos de registro

Eles não são burocracia: são a prova de autoria e, na prática, o material de estudo mais útil que você vai levar desta atividade. Escreva-os **enquanto** trabalha.

7.1 PROMPTS.md

Os prompts que produziram código no projeto — não todos, os que **funcionaram**.

```
## Etapa 2 – CardProduto

**Prompt** (anexei a imagem do cartão):
> Você é uma pessoa desenvolvedora React sênior. Contexto: React 18 + Vite...
> Escreva SÓ o CardProduto.jsx. Restrições: componente de função, ...

**Ferramenta**: Claude · **Tentativas até funcionar**: 2

**0 que voltou**: componente correto, mas usando `produto.image`.
**0 que eu mudei**: troquei para `thumbnail` e ajustei o cálculo do preço,
que vinha sem a conversão para real.
```

7.2 DIARIO-DA-IA.md

Pelo menos cinco entradas. Cada uma é um erro que a IA cometeu, como você percebeu e o que fez. Um diário vazio significa uma de duas coisas: ou você não usou IA, ou não leu o que ela escreveu — e a segunda é bem mais provável.

```
## Erro 3 – laço infinito de requisições

**Onde**: ListaProdutos.jsx, etapa 3
**0 que a IA escreveu**: um useEffect sem array de dependências.
**Como percebi**: a aba Rede do navegador não parava de acumular
requisições para /products, e o ventilador do notebook ligou.
**Por que acontece**: sem array, o efeito roda depois de toda
renderização; ele chama setState, que renderiza de novo, que roda
o efeito de novo.
**Correção**: array [pagina, busca, categoria] – as coisas que, quando
mudam, realmente exigem buscar de novo.
**0 que aprendi**: o array de dependências não é enfeite; é a pergunta
"quando isto precisa acontecer de novo?".
```

8. Checklist antes de considerar pronto

- Clonei o repositório em outra pasta, rodei o README **ao pé da letra** e o app subiu;
- As três telas batem com os mockups em hierarquia, espaçamento e estados;
- Os quatro estados aparecem de verdade — eu testei desligando a rede;
- Nenhum aviso de **key** e nenhum erro vermelho no console;
- Busca, categoria e página estão na URL e sobrevivem ao F5;
- O carrinho soma quantidade em vez de duplicar linha, e os totais fecham na calculadora;
- As três larguras funcionam, sem rolagem horizontal;
- **npm run build** termina sem erro;

- **PROMPTS.md** e **DIARIO-DA-IA.md** estão preenchidos, com cinco erros reais;
- Abri um arquivo qualquer do projeto ao acaso e consegui explicar o que ele faz.

9. Erros comuns: diagnóstico rápido

Sintoma	Causa provável	Correção
Tela branca e um erro vermelho no console	erro de renderização — quase sempre <code>.map</code> em algo que não é vetor	leia a primeira linha do erro; ela diz o arquivo e a linha
"products is not defined"	desestruturação da resposta errada	<code>const { products } = await response.json()</code>
Todas as imagens quebradas	campo errado (<code>image</code> em vez de <code>thumbnail</code>)	confira o JSON real no navegador
Requisições sem parar na aba Rede	<code>useEffect</code> sem array de dependências	declare as dependências reais
A busca dispara a cada tecla	falta o atraso (debounce)	espere 400 ms sem digitação antes de chamar
Filtro aplicado e a página continua na 5	página não voltou para 1	ao mudar busca ou categoria, <code>pagina = 1</code>
F5 na rota de detalhe dá 404 no deploy	servidor não redireciona para o index	configurar reescrita para SPA na Vercel/Netlify
Total do carrinho errado depois de remover	total guardado em estado	calcular a partir dos itens na renderização
Estilo não aplica	<code>class</code> em vez de <code>className</code> , ou seletor mais específico vencendo	corrigir o atributo; resolver especificidade sem <code>!important</code>
Rolagem horizontal no celular	largura fixa em px em algum contêiner	<code>max-width: 100%</code> e medidas relativas
Erro de CORS	endereço da API digitado errado	a DummyJSON libera CORS; o erro costuma ser URL torta

PARA SABER MAIS — Materiais

- **mockups/** — as seis telas em alta resolução, ao lado deste PDF;
- **Módulos 5 a 9** — props e listas, estado e eventos, `useEffect` e dados remotos, serviços, e rotas com React Router: é tudo que esta atividade cobra;
- **Projeto Pokédex** (módulo 7) — o mesmo caminho de `useEffect` + API, com o passo a passo já andado;
- **dummyjson.com/docs/products** — a documentação da API;
- **react.dev** — a referência oficial de hooks; use-a para conferir o que a IA afirmar.